

# 2026-05-06-p1-f17-smart-file-editing

## P1-F17 — Smart File Editing Implementation Plan

**For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (- [ ]) syntax for tracking.

**Goal:** Ship real, end-to-end **search-replace block editing** for the HelixCode CLI agent. F17 adds a `smart_edit` tool that consumes one or more **search-replace blocks** in a single string payload, applies them transactionally to one or more files via the existing F08 multiedit transaction layer, and returns both the updated file content (re-read from disk) and a unified diff so the caller can self-check that the change actually landed. A `/edit slash` command (`status / diff / dry-run <path> / commit <path>`) provides inspection and a one-shot dry-run / commit from a file containing blocks. **No cobra subcommand** (Q5=A). **No mtime/hash strict conflict check** (Q4=B; lenient — re-search at apply time). **No new external dependencies** (\$Tech Stack). **All-or-nothing across the whole prompt** (Q3=B; one failed block aborts the entire commit and writes nothing). Multi-file atomicity inherits from F08 multiedit's transactional Commit + backup-restore rollback.

**Architecture:** New `internal/tools/smartedit/` package with `types.go` (`EditBlock` + `EditPlan` + `BlockResult` + `EditResult` + `SmartEditResult` + marker constants + error sentinels + size limits), `parser.go` (state-machine tokeniser over the prompt's lines; path-line stickiness across consecutive blocks for the same file), `applier.go` (re-search each SEARCH literal in current content; ambiguous → fail; not-found → fail; in-order block application so a later block sees an earlier replace), `diff.go` (5-line wrapper around `multiedit.DiffManager.GeneratedDiff` so we re-use the F08 LCS impl), `binary_detect.go` (NUL-byte heuristic; refuse binaries), `smart_edit_tool.go` (Tool impl wrapping parser + applier + multiedit transaction; tracks `lastResult` for the slash command's status/diff inspection; exposes `DryRun` + `Commit` for the slash). Slash command at `internal/commands/edit_command.go` mirroring F14 / `sandbox` and F16 / `telemetry` shape (defines `SmartEditInspector` interface in the commands package so the slash is testable with a fake). Two existing files get tiny additions: `internal/tools/registry.go` (one line: register `smart_edit` in the tool map) and `cmd/cli/main.go` (three lines: construct the tool, register it, register the slash).

**Tech Stack:** Go 1.26, testify v1.11, zap (already in `go.mod`) — already present. **NO new external deps.** Brief justification: (1) parser is line-by-line text → `stdlib bufio.Scanner` on a string suffices; (2) applier is `strings.Index / strings.Count / strings.Replace` per block; (3) diff is delegated to the existing F08 `multiedit.DiffManager` (LCS-based unified diff already in production — re-use is the explicit anti-bluff choice); (4) atomicity is delegated to F08 `MultiFileEditor` (already in production); (5) binary detect is the standard “first 8 KiB contains NUL” heuristic. `go mod tidy` after T02 must produce **zero new entries** in `go.sum` — if it doesn't, the implementation drifted from the spec.

**Spec:** `docs/superpowers/specs/2026-05-06-p1-f17-smart-file-editing-design.md` (commit `fa77f09`)

**Working directory for go commands:** `helix_code/`. Git from meta-repo root.

### Anti-bluff smoke (FULL 4-term applied to F17 surface):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
placeholder" \
internal/tools/smartedit internal/commands/edit_command.go \
&& echo BLUFF || echo clean
```

Must always print clean.

**Anti-bluff hot zone:** §5.2 of the spec — smart\_edit can degenerate into a no-op or a partial-write in four ways: (a) Applied=true but file unchanged on disk (multiedit Commit failed silently or the post-write re-read was skipped); (b) SEARCH not found but tool returned success (the applier silently skipped the block); (c) multi-file commit reports success but only some files were written (atomicity broken — multiedit rollback didn't restore); (d) returned Diff doesn't match what's actually on disk (computed from the planned post-content, not the re-read). The eight real-execution criteria (Applied=true ⇒ disk changed; Applied=false ⇒ disk unchanged; multi-file atomic; partial-failure rollback; diff exactness; ambiguous rejected; binary refused; no diff text at INFO log level) are each tested with both unit assertions AND a Challenge phase. The Challenge harness uses sha-256 before-and-after as positive evidence — disk-state mismatch is a hard Challenge failure. Absence-of-error is NEVER acceptable.

**Why this is consequential:** every coding agent's primary destructive operation is editing files. A smart-edit tool that quietly says "applied" without actually writing is the worst class of bug — the LLM proceeds to its next step on a false premise. F17's discriminating tests are: (i) the Challenge's PARTIAL-FAILURE-ROLLBACK phase (chmod 0500 on file 3's parent dir; assert all 5 files' sha-256 values are byte-identical to their pre-content), and (ii) the unit-level TestSmartEditTool\_AppliedFalse\_FileUnchanged (deliberately unfindable SEARCH; assert sha-before == sha-after). Both must produce positive evidence; neither can be satisfied by absence-of-error.

---

## Task list



P1-F17-T01 — bootstrap evidence + advance PROGRESS to F17



P1-F17-T02 — internal/tools/smartedit/types.go: EditBlock + EditPlan + BlockResult + EditResult + SmartEditResult + marker constants + error sentinels + size limits (TDD)



P1-F17-T03 — internal/tools/smartedit/parser.go: parse SEARCH/ REPLACE blocks from prompt string; path-line stickiness; line-number tracking (TDD)



P1-F17-T04 — internal/tools/smartedit/applier.go + binary\_detect.go: in-order block apply with lenient re-search; ambiguity detection; binary refusal (TDD)



P1-F17-T05 — internal/tools/smartedit/diff.go: pure-Go unified-diff wrapper around multiedit.DiffManager.GeneratedDiff; assert byte-exact match with system diff -u (TDD)



P1-F17-T06 — `internal/tools/smartedit/smart_edit_tool.go`: Tool impl wrapping parser + applier + multiedit transaction; post-write re-read; `DryRun / Commit / LastResult / LastBlocks` (TDD with real `MultiFileEditor` against tempdirs)



P1-F17-T07 — `internal/commands/edit_command.go`: `/edit slash (status / diff / dry-run / commit)`; `SmartEditInspector` interface in commands package (TDD with fake)



P1-F17-T08 — `cmd/cli/main.go` wiring + `internal/tools/registry.go` registration + integration tests (`-tags=integration`; always-runs against real tempdir)



P1-F17-T09 — Challenge harness (7-phase: SINGLE-FILE-SUCCESS + SEARCH-NOT-FOUND-REJECTED + MULTI-FILE-ATOMIC + PARTIAL-FAILURE-ROLLBACK + DIFF-EXACTNESS + AMBIGUOUS-REJECTED + BINARY-REFUSED) with sha-256 positive evidence per phase



P1-F17-T10 — Feature 17 close-out + push 4 remotes non-force

---

## Task 1: Bootstrap

Append F17 evidence section header (spec `fa77f09`), update PROGRESS current focus to F17, insert F17 task list (10 items) after F16's. Confirm `06_phase_1_evidence.md` has an F17 anchor.

Commit: docs(P1-F17-T01): bootstrap Phase 1 / Feature 17 evidence + advance PROGRESS.

---

## Task 2: types.go (TDD)

**Files:** new `helix_code/internal/tools/smartedit/types.go`, new `helix_code/internal/tools/smartedit/types_test.go`.

Define: - Marker constants (`searchMarker = "<<<<<<< SEARCH"`, `dividerMarker = "====="`, `replaceMarker = ">>>>>>> REPLACE"`). - `EditBlock{Path, Search, Replace string; Line int}`. - `EditPlan map[string][]EditBlock`. - `BlockResult{Block EditBlock; Applied bool; Reason string}`. - `EditResult{Path string; Applied bool; NewContent string; Diff string; Blocks []BlockResult; Error error}`. - `SmartEditResult{Files []EditResult; Applied bool; Diff string; DurationMs int64}`. - Error sentinels (`ErrParseEmpty`, `ErrParseMalformed`, `ErrParseNoPath`, `ErrSearchNotFound`, `ErrSearchAmbiguous`, `ErrEmptySearch`, `ErrBinaryFile`, `ErrApplyFailed`, `ErrFileTooLarge`, `ErrPromptTooLarge`). - Size limits (`MaxPromptBytes = 4 MiB`, `MaxFileBytes = 4 MiB`).

Failing tests FIRST:

```
func TestMarkerConstants_ExactValues(t *testing.T) {
    require.Equal(t, "<<<<<<< SEARCH", searchMarker)
```

```

    require.Equal(t, "=====", dividerMarker)
    require.Equal(t, ">>>>>> REPLACE", replaceMarker)
}

func TestErrorSentinels_DistinctErrorsIs(t *testing.T) {
    for _, e := range []error{
        ErrParseEmpty, ErrParseMalformed, ErrParseNoPath,
        ErrSearchNotFound, ErrSearchAmbiguous, ErrEmptySearch,
        ErrBinaryFile, ErrApplyFailed, ErrFileTooLarge,
        ErrPromptTooLarge,
    } {
        wrapped := fmt.Errorf("wrapped: %w", e)
        require.ErrorIs(t, wrapped, e)
    }
}

func TestSizeLimits_NonZero(t *testing.T) {
    require.Greater(t, MaxPromptBytes, 0)
    require.Greater(t, MaxFileBytes, 0)
}

func TestEditResult_FieldsZeroValueOK(t *testing.T) {
    r := EditResult{}
    require.False(t, r.Applied)
    require.Empty(t, r.NewContent)
    require.Empty(t, r.Diff)
}

```

Subject: feat(P1-F17-T02): EditBlock + EditResult + marker constants + error sentinels.

---

### Task 3: parser.go (TDD)

**Files:** new helix\_code/internal/tools/smartedit/parser.go, new helix\_code/internal/tools/smartedit/parser\_test.go.

parser.go exports: - Parse(prompt string) ([]EditBlock, error) — state-machine over lines: scanPath → on marker → scanSearch → on divider → scanReplace → on terminator → emit block, back to scanPath (path-line sticky for next block targeting same file). Reject malformed prompts (missing markers, marker without preceding path, EOF mid-block).

Failing tests FIRST:

```

const blocksOK = "" +
    "path/a.go\n" +
    "<<<<<< SEARCH\n" +
    "old line\n" +
    "=====\n" +

```

```

    "new line\n" +
    ">>>>>> REPLACE\n"

func TestParse_SingleBlock_OK(t *testing.T) {
    bs, err := Parse(blocksOK)
    require.NoError(t, err)
    require.Len(t, bs, 1)
    require.Equal(t, "path/a.go", bs[0].Path)
    require.Equal(t, "old line", bs[0].Search)
    require.Equal(t, "new line", bs[0].Replace)
    require.Equal(t, 2,
        bs[0].Line) // SEARCH marker is line 2
}

func TestParse_MultipleBlocksSameFile_PathSticky(t *testing.T) {
    in := blocksOK + // first block
        "<<<<<< SEARCH\n" +
        "old2\n" +
        "=====\n" +
        "new2\n" +
        ">>>>>> REPLACE\n"
    bs, err := Parse(in)
    require.NoError(t, err)
    require.Len(t, bs, 2)
    require.Equal(t, "path/a.go", bs[0].Path)
    require.Equal(t, "path/a.go", bs[1].Path) // sticky
}

func TestParse_MultipleFiles_OK(t *testing.T) {
    in := blocksOK + "path/b.go\n" +
        "<<<<<< SEARCH\n=====\n>>>>>> REPLACE\n"
    bs, err := Parse(in)
    require.NoError(t, err)
    require.Len(t, bs, 2)
    require.Equal(t, "path/a.go", bs[0].Path)
    require.Equal(t, "path/b.go", bs[1].Path)
}

func TestParse_EmptyPrompt_ErrParseEmpty(t *testing.T) {
    _, err := Parse("")
    require.ErrorIs(t, err, ErrParseEmpty)
}

func TestParse_NoBlocks_ErrParseEmpty(t *testing.T) {
    _, err := Parse("just some text\nwith no markers\n")
    require.ErrorIs(t, err, ErrParseEmpty)
}

```

```

func TestParse_SearchWithoutPath_ErrParseNoPath(t *testing.T) {
    in := "<<<<<< SEARCH\nfoo\n=====\nbar\n>>>>>> REPLACE\n"
    _, err := Parse(in)
    require.ErrorIs(t, err, ErrParseNoPath)
}

func TestParse_MissingDivider_ErrParseMalformed(t *testing.T) {
    in := "p\n<<<<<< SEARCH\nfoo\n>>>>>> REPLACE\n"
    _, err := Parse(in)
    require.ErrorIs(t, err, ErrParseMalformed)
}

func TestParse_MissingTerminator_ErrParseMalformed(t *testing.T) {
    in := "p\n<<<<<< SEARCH\nfoo\n=====\nbar\n"
    _, err := Parse(in)
    require.ErrorIs(t, err, ErrParseMalformed)
}

func TestParse_LineNumberRecordedInBlock(t *testing.T) {
    in := "\n\npath/a.go\n<<<<<< SEARCH\nx\n=====\ny\n>>>>>> REPLACE\n"
    bs, _ := Parse(in)
    require.Equal(t, 4, bs[0].Line)
}

func TestParse_RejectsCol0MarkerInsideBlock_DocumentingLimitation(t
    *testing.T) {
    // Documents v1 limitation: SEARCH body containing the marker
    // triplet at col 0
    // is parsed as a malformed prompt. v2 may add escaping.
    in := "p\n<<<<<< SEARCH\n<<<<<< SEARCH\n=====\nbar\n>>>>>> REPLACE\n"
    _, err := Parse(in)
    require.Error(t, err)
}

```

Subject: feat(P1-F17-T03): SmartEditParser with path-line stickiness + line tracking.

---

## Task 4: applier.go + binary\_detect.go (TDD)

**Files:** new helix\_code/internal/tools/smartedit/applier.go, new helix\_code/internal/tools/smartedit/applier\_test.go, new helix\_code/internal/tools/smartedit/binary\_detect.go, new helix\_code/internal/tools/smartedit/binary\_detect\_test.go.

applier.go:-Apply(content string, blocks []EditBlock) (string, []BlockResult, error) — for each block, strings.Count(current, b.Search) decides outcome: 0 → ErrSearchNotFound; 1 → strings.Replace and continue; >1 → ErrSearchAmbiguous. Empty Search → ErrEmptySearch. First block-level failure aborts the file (returns the failing block result + sentinel error). - findExactlyOnce(content, search string) (int, error) — used internally; exposed for tests.

binary\_detect.go:-IsBinary(content []byte) bool — bytes.IndexByte(content[:min(8192, len(content))], 0x00) >= 0.

Failing tests FIRST (applier\_test.go):

```
func TestApply_SingleBlock_Replaces(t *testing.T) {
    out, results, err := Apply("hello world",
        []EditBlock{{Path:"x", Search:"world", Replace:"there"}})
    require.NoError(t, err)
    require.Equal(t, "hello there", out)
    require.Len(t, results, 1)
    require.True(t, results[0].Applied)
}
```

```
func TestApply_SearchNotFound_ErrSearchNotFound_AbortsFile(t
    *testing.T) {
    out, results, err := Apply("hello world",
        []EditBlock{{Path:"x", Search:"xyz", Replace:"abc"}})
    require.ErrorIs(t, err, ErrSearchNotFound)
    require.Equal(t, "hello world", out) // unchanged
    require.Len(t, results, 1)
    require.False(t, results[0].Applied)
    require.Contains(t, results[0].Reason, "not found")
}
```

```
func TestApply_AmbiguousSearch_ErrSearchAmbiguous(t *testing.T) {
    out, results, err := Apply("xx xx xx",
        []EditBlock{{Path:"y", Search:"xx", Replace:"yy"}})
    require.ErrorIs(t, err, ErrSearchAmbiguous)
    require.Equal(t, "xx xx xx", out)
    require.Contains(t, results[0].Reason, "ambiguous")
}
```

```
func TestApply_EmptySearch_ErrEmptySearch(t *testing.T) {
    _, _, err := Apply("anything", []EditBlock{{Path:"z",
        Search:"", Replace:"x"}})
    require.ErrorIs(t, err, ErrEmptySearch)
}
```

```
func TestApply_OrderRespected_LaterBlockSeesEarlierReplace(t
    *testing.T) {
    blocks := []EditBlock{
```

```

        {Path:"f", Search:"A", Replace:"B"},
        {Path:"f", Search:"B", Replace:"C"},
    }
    out, _, err := Apply("A", blocks)
    require.NoError(t, err)
    require.Equal(t, "C", out) // A→B then B→C
}

func TestApply_FirstFailureAborts_LaterBlocksUnattempted(t
    *testing.T) {
    blocks := []EditBlock{
        {Path:"f", Search:"NOPE", Replace:"x"}, // fails
        {Path:"f", Search:"hello", Replace:"y"}, // would succeed
    }
    out, results, err := Apply("hello world", blocks)
    require.ErrorIs(t, err, ErrSearchNotFound)
    require.Equal(t, "hello world", out)
    require.Len(t, results, 1) // only first attempted
}

func TestApply_MultilineSearchReplace_OK(t *testing.T) {
    in := "first\nsecond\nthird\n"
    bs := []EditBlock{{Path:"f", Search:"first\nsecond",
        Replace:"FIRST\nSECOND"}}
    out, _, err := Apply(in, bs)
    require.NoError(t, err)
    require.Equal(t, "FIRST\nSECOND\nthird\n", out)
}

```

Failing tests FIRST (binary\_detect\_test.go):

```

func TestIsBinary_DetectsNUL(t *testing.T) {
    require.True(t, IsBinary([]byte{'a', 'b', 0, 'c'}))
}
func TestIsBinary_TextFile_False(t *testing.T) {
    require.False(t, IsBinary([]byte("package
        main\n\nfunc main() {}\n")))
}
func TestIsBinary_EmptyFile_False(t *testing.T) {
    require.False(t, IsBinary(nil))
    require.False(t, IsBinary([]byte{}))
}
func TestIsBinary_NULBeyond8KiB_False(t *testing.T) {
    buf := make([]byte, 9000)
    for i := range buf { buf[i] = 'a' }
    buf[8500] = 0
    require.False(t, IsBinary(buf)) // only first 8 KiB scanned
}

```



Subject: feat(P1-F17-T04): SmartEditApplier + binary detect; lenient re-search + ambiguity detection.

---

## Task 5: diff.go (TDD)

**Files:** new helix\_code/internal/tools/smartedit/diff.go, new helix\_code/internal/tools/smartedit/diff\_test.go.

diff.go:

```
import "dev.helix.code/internal/tools/multiedit"

// UnifiedDiff returns a unified-format diff between old and new
// for path.
// Wraps the existing F08 multiedit.DiffManager.GenerateDiff so
// the codebase
// has exactly one LCS-based unified-diff implementation.
func UnifiedDiff(oldContent, newContent, path string) string {
    dm := multiedit.NewDiffManager(multiedit.FormatUnified)
    d, err := dm.GenerateDiff([]byte(oldContent),
        []byte(newContent), path)
    if err != nil || d == nil {
        return ""
    }
    return d.Unified
}
```

Failing tests FIRST:

```
func TestUnifiedDiff_NoChange_EmptyOrNoOpDiff(t *testing.T) {
    d := UnifiedDiff("hello\n", "hello\n", "f.txt")
    // multiedit may return an empty unified diff for unchanged
    // content; either is fine
    require.NotContains(t, d, "+hello")
    require.NotContains(t, d, "-hello")
}

func TestUnifiedDiff_ReplaceLine_HasAddAndRemove(t *testing.T) {
    d := UnifiedDiff("foo\nbar\n", "foo\nBAR\n", "f.txt")
    require.Contains(t, d, "-bar")
    require.Contains(t, d, "+BAR")
}

func TestUnifiedDiff_MatchesDiffU(t *testing.T) {
    if _, err := exec.LookPath("diff"); err != nil {
        t.Skip("SKIP-OK: P1-F17 system `diff` not installed")
    }
    dir := t.TempDir()
```

```

pre := filepath.Join(dir, "pre.txt")
post := filepath.Join(dir, "post.txt")
require.NoError(t, os.WriteFile(pre, []byte("a\nb\nc\n"),
    0o644))
require.NoError(t, os.WriteFile(post, []byte("a\nB\nc\n"),
    0o644))
out, _ := exec.Command("diff", "-u", pre,
    post).CombinedOutput()
got := UnifiedDiff("a\nb\nc\n", "a\nB\nc\n", "f.txt")
// Compare structurally: both must contain the same -/+/@@
// lines for the changed hunk
require.Contains(t, got, "-b")
require.Contains(t, got, "+B")
require.Contains(t, string(out), "-b")
require.Contains(t, string(out), "+B")
}

```

Subject: feat(P1-F17-T05): unified-diff wrapper re-using F08  
 multiedit DiffManager.

---

## Task 6: smart\_edit\_tool.go (TDD; real disk via tempdirs + real MultiFileEditor)

**Files:** new helix\_code/internal/tools/smartedit/smart\_edit\_tool.go, new  
 helix\_code/internal/tools/smartedit/smart\_edit\_tool\_test.go.

Implementation outline:

```

type SmartEditTool struct {
    multiEdit *multiedit.MultiFileEditor
    logger     *zap.Logger

    mu          sync.RWMutex
    lastResult  *SmartEditResult
    lastBlocks []EditBlock
}

func NewSmartEditTool(me *multiedit.MultiFileEditor, logger
    *zap.Logger) *SmartEditTool {
    if logger == nil { logger = zap.NewNop() }
    return &SmartEditTool{multiEdit: me, logger: logger}
}

func (t *SmartEditTool) Name() string { return "smart_edit" }
func (t *SmartEditTool) Description() string { return "Apply
    SEARCH/REPLACE blocks to one or more files atomically." }
func (t *SmartEditTool) Category() tools.ToolCategory { return
    tools.CategoryMultiEdit }

```

```

func (t *SmartEditTool) Schema() tools.ToolSchema { /* one
    required string param: prompt */ }
func (t *SmartEditTool) Validate(params map[string]interface{})
    error { /* require prompt:string */ }

func (t *SmartEditTool) Execute(ctx context.Context, params
    map[string]interface{}) (interface{}, error) {
    prompt, _ := params["prompt"].(string)
    return t.commit(ctx, prompt, false /* dryRun */)
}
func (t *SmartEditTool) DryRun(ctx context.Context, prompt
    string) (*SmartEditResult, error) {
    return t.commit(ctx, prompt, true)
}
func (t *SmartEditTool) Commit(ctx context.Context, prompt
    string) (*SmartEditResult, error) {
    return t.commit(ctx, prompt, false)
}
func (t *SmartEditTool) LastResult() *SmartEditResult { /* RLock
    */ }
func (t *SmartEditTool) LastBlocks() []EditBlock { /* RLock
    */ }

// commit is the shared pipeline: parse → group → in-memory apply
// →
// (if !dryRun) multiedit transaction → re-read → diff. dryRun
// returns
// the planned diff against the planned post-content WITHOUT
// writing.
func (t *SmartEditTool) commit(ctx context.Context, prompt
    string, dryRun bool) (*SmartEditResult, error) { ... }

```

Failing tests FIRST (every test uses t.TempDir() + real disk):

```

func newTestTool(t *testing.T) (*SmartEditTool, string) {
    t.Helper()
    dir := t.TempDir()
    cfg := multiedit.DefaultConfig()
    cfg.WorkspaceRoot = dir
    cfg.RequirePreview = false // we drive Preview ourselves
    me, err :=
        multiedit.NewMultiFileEditor(multiedit.WithConfig(cfg))
    require.NoError(t, err)
    return NewSmartEditTool(me, zap.NewNop()), dir
}

func sha(t *testing.T, p string) string {
    t.Helper()
    b, err := os.ReadFile(p); require.NoError(t, err)

```

```

    h := sha256.Sum256(b); return hex.EncodeToString(h[:])
}

func TestSmartEditTool_AppliedTrue_ImpliesFileChanged(t
    *testing.T) {
    tool, dir := newTestTool(t)
    p := filepath.Join(dir, "a.go")
    require.NoError(t, os.WriteFile(p, []byte("package
        x\nvar foo = 1\n"), 0o644))
    pre := sha(t, p)

    prompt := p + "\n<<<<<< SEARCH\nvar foo = 1\n=====\nvar
        foo = 42\n>>>>>> REPLACE\n"
    out, err := tool.Execute(context.Background(),
        map[string]interface{}{"prompt": prompt})
    require.NoError(t, err)
    res := out.(*SmartEditResult)
    require.True(t, res.Applied)
    require.NotEqual(t, pre, sha(t, p))
}

func TestSmartEditTool_AppliedFalse_FileUnchanged(t *testing.T) {
    tool, dir := newTestTool(t)
    p := filepath.Join(dir, "a.go")
    require.NoError(t, os.WriteFile(p, []byte("package x\n"),
        0o644))
    pre := sha(t, p)

    prompt := p + "\n<<<<<< SEARCH\nNOT IN
        FILE\n=====\nx\n>>>>>> REPLACE\n"
    _, err := tool.Execute(context.Background(),
        map[string]interface{}{"prompt": prompt})
    require.Error(t, err)
    require.Equal(t, pre, sha(t, p)) // file untouched
}

func TestSmartEditTool_NewContentMatchesDisk(t *testing.T) {
    tool, dir := newTestTool(t)
    p := filepath.Join(dir, "a.go")
    require.NoError(t, os.WriteFile(p, []byte("foo\n"), 0o644))
    prompt := p + "\n<<<<<< SEARCH\nfoo\n=====\nbar\n>>>>>>
        REPLACE\n"
    out, _ := tool.Execute(context.Background(),
        map[string]interface{}{"prompt": prompt})
    res := out.(*SmartEditResult)
    onDisk, _ := os.ReadFile(p)
    require.Equal(t, string(onDisk), res.Files[0].NewContent)
}

```

```

func TestSmartEditTool_MultiFile_AllOrNothing_OneBlockFails(t
    *testing.T) {
    tool, dir := newTestTool(t)
    a := filepath.Join(dir, "a.go"); b := filepath.Join(dir,
        "b.go")
    require.NoError(t, os.WriteFile(a, []byte("foo\n"), 0o644))
    require.NoError(t, os.WriteFile(b, []byte("baz\n"), 0o644))
    preA, preB := sha(t, a), sha(t, b)
    prompt :=
        a + "\n<<<<<<< SEARCH\nfoo\n=====\nF00\n>>>>>>>
        REPLACE\n" +
        b + "\n<<<<<<< SEARCH\nNOPE\n=====\nX\n>>>>>>>
        REPLACE\n"
    _, err := tool.Execute(context.Background(),
        map[string]interface{}{"prompt": prompt})
    require.Error(t, err)
    require.Equal(t, preA, sha(t, a)) // a NOT written even
        though its block would succeed
    require.Equal(t, preB, sha(t, b))
}

```

```

func TestSmartEditTool_BinaryFile_Refused(t *testing.T) {
    tool, dir := newTestTool(t)
    p := filepath.Join(dir, "bin.dat")
    require.NoError(t, os.WriteFile(p, []byte{'a', 0, 'b'},
        0o644))
    pre := sha(t, p)
    prompt := p + "\n<<<<<<< SEARCH\na\n=====\nA\n>>>>>>>
        REPLACE\n"
    _, err := tool.Execute(context.Background(),
        map[string]interface{}{"prompt": prompt})
    require.Error(t, err)
    require.ErrorIs(t, err, ErrBinaryFile)
    require.Equal(t, pre, sha(t, p))
}

```

```

func TestSmartEditTool_DryRun_DoesNotWrite(t *testing.T) {
    tool, dir := newTestTool(t)
    p := filepath.Join(dir, "a.go")
    require.NoError(t, os.WriteFile(p, []byte("foo\n"), 0o644))
    pre := sha(t, p)
    prompt := p + "\n<<<<<<< SEARCH\nfoo\n=====\nF00\n>>>>>>>
        REPLACE\n"
    res, err := tool.DryRun(context.Background(), prompt)
    require.NoError(t, err)
    require.True(t, res.Applied) // planned-apply
        succeeded in-memory
}

```

```

require.NotEmpty(t, res.Diff)
require.Equal(t, pre, sha(t, p)) // BUT the file is
    byte-identical
}

func TestSmartEditTool_LastResultPopulated(t *testing.T) {
    tool, dir := newTestTool(t)
    p := filepath.Join(dir, "a.go")
    require.NoError(t, os.WriteFile(p, []byte("x\n"), 0o644))
    prompt := p + "\n<<<<<< SEARCH\nx\n=====\ny\n>>>>>>
        REPLACE\n"
    _, _ = tool.Execute(context.Background(),
        map[string]interface{}{"prompt": prompt})
    require.NotNil(t, tool.LastResult())
    require.Len(t, tool.LastBlocks(), 1)
}

func TestSmartEditTool_DoesNotLogDiffAtInfo(t *testing.T) {
    obs, logs := observer.New(zapcore.InfoLevel)
    logger := zap.New(obs)
    cfg := multiedit.DefaultConfig(); cfg.RequirePreview = false
    me, _ :=
        multiedit.NewMultiFileEditor(multiedit.WithConfig(cfg))
    tool := NewSmartEditTool(me, logger)
    dir := t.TempDir()
    p := filepath.Join(dir, "a.go")
    require.NoError(t, os.WriteFile(p, []byte("SECRET_TOKEN=sk-
        deadbeef\n"), 0o644))
    prompt := p + "\n<<<<<< SEARCH\nSECRET_TOKEN=sk-
        deadbeef\n=====\nSECRET_TOKEN=sk-rotated\n>>>>>>
        REPLACE\n"
    _, _ = tool.Execute(context.Background(),
        map[string]interface{}{"prompt": prompt})
    for _, e := range logs.All() {
        require.NotContains(t, e.Message + e.ContextMap()
            ["diff"].(string), "sk-deadbeef")
    }
}

func TestSmartEditTool_LargePrompt_ErrPromptTooLarge(t
    *testing.T) { /* prompt > MaxPromptBytes */ }
func TestSmartEditTool_LargeFile_ErrFileTooLarge(t
    *testing.T) { /* file > MaxFileBytes */ }

```

Subject: feat(P1-F17-T06): SmartEditTool with multiedit transaction + post-write re-read + diff.

---

## Task 7: /edit slash command (TDD)

**Files:** new helix\_code/internal/commands/edit\_command.go, new helix\_code/internal/commands/edit\_command\_test.go.

Mirrors F14 /sandbox and F16 /telemetry shape: defines SmartEditInspector interface in the commands package so the slash is testable with a fake.

```
type SmartEditInspector interface {
    LastResult() *smartedit.SmartEditResult
    LastBlocks() []smartedit.EditBlock
    DryRun(ctx context.Context, prompt string)
        (*smartedit.SmartEditResult, error)
    Commit(ctx context.Context, prompt string)
        (*smartedit.SmartEditResult, error)
}

type EditCommand struct { insp SmartEditInspector }

func NewEditCommand(insp SmartEditInspector) *EditCommand
func (c *EditCommand) Name() string { return "edit" }
func (c *EditCommand) Description() string { return "Inspect
    last smart_edit attempt or run a dry-run / commit from a
    blocks file." }
func (c *EditCommand) Usage() string { return "/edit
    [status|diff|dry-run <path>|commit <path>]" }
func (c *EditCommand) Execute(ctx context.Context, cc
    *CommandContext) (*CommandResult, error) {
    if c.insp == nil { return &CommandResult{Success: false,
        Output: "smart_edit unavailable"}, nil }
    sub := "status"
    if len(cc.Args) > 0 { sub = cc.Args[0] }
    switch sub {
    case "status": return c.handleStatus(), nil
    case "diff": return c.handleDiff(), nil
    case "dry-run": if len(cc.Args)<2 { return
        &CommandResult{Success:false, Output:c.Usage()}, nil }
        return c.handleDryRun(ctx, cc.Args[1])
    case "commit": if len(cc.Args)<2 { return
        &CommandResult{Success:false, Output:c.Usage()}, nil }
        return c.handleCommit(ctx, cc.Args[1])
    default: return &CommandResult{Success:false,
        Output:c.Usage()}, nil
    }
}
```

Failing tests FIRST:

```

type fakeInspector struct {
    last      *smartedit.SmartEditResult
    blocks    []smartedit.EditBlock
    dryFn      func(ctx context.Context, prompt string)
              (*smartedit.SmartEditResult, error)
    commitFn   func(ctx context.Context, prompt string)
              (*smartedit.SmartEditResult, error)
}

func (f *fakeInspector) LastResult() *smartedit.SmartEditResult
    { return f.last }
func (f *fakeInspector) LastBlocks() []smartedit.EditBlock
    { return f.blocks }
func (f *fakeInspector) DryRun(ctx context.Context, prompt
    string) (*smartedit.SmartEditResult, error) {
    return f.dryFn(ctx, prompt)
}
func (f *fakeInspector) Commit(ctx context.Context, prompt
    string) (*smartedit.SmartEditResult, error) {
    return f.commitFn(ctx, prompt)
}

func TestEditCommand_NilInspector_ReportsUnavailable(t
    *testing.T) {
    c := NewEditCommand(nil)
    res, _ := c.Execute(context.Background(), &CommandContext{})
    require.False(t, res.Success)
    require.Contains(t, res.Output, "unavailable")
}

func TestEditCommand_Status_NoAttempt_ReportsNone(t *testing.T) {
    c := NewEditCommand(&fakeInspector{})
    res, _ := c.Execute(context.Background(), &CommandContext{})
    require.Contains(t, res.Output, "no smart_edit attempts")
}

func TestEditCommand_Status_LastAttempt_RendersTable(t
    *testing.T) {
    f := &fakeInspector{last:
        &smartedit.SmartEditResult{Applied: true,
            Files: []smartedit.EditResult{{Path: "a.go",
                Applied: true, Diff: "@@ -1 +1 @@\n-x\n+y\n"}}}}
    c := NewEditCommand(f)
    res, _ := c.Execute(context.Background(),
        &CommandContext{Args: []string{"status"}})
    require.Contains(t, res.Output, "a.go")
}

```



```

func TestEditCommand_Diff_LastAttempt_PrintsDiff(t *testing.T)
    { /* ... */ }

func TestEditCommand_DryRun_FileNotFound_ReportsError(t
    *testing.T) {
    c := NewEditCommand(&fakeInspector{})
    res, _ := c.Execute(context.Background(),
        &CommandContext{Args: []string{"dry-run", "/no/such/
        file"}})
    require.False(t, res.Success)
    require.Contains(t, res.Output, "no such file")
}

func TestEditCommand_DryRun_ValidPrompt_ReturnsDiff_NoWrite(t
    *testing.T) {
    dir := t.TempDir()
    blocksFile := filepath.Join(dir, "blocks.txt")
    require.NoError(t, os.WriteFile(blocksFile,
        []byte("p\n<<<<<< SEARCH\nx\n=====\\ny\n>>>>>>
        REPLACE\\n"), 0o644))
    f := &fakeInspector{dryFn: func(ctx context.Context, prompt
        string) (*smartedit.SmartEditResult, error) {
        return &smartedit.SmartEditResult{Applied: true, Diff:
        "diff-text"}, nil
    }}
    c := NewEditCommand(f)
    res, _ := c.Execute(context.Background(),
        &CommandContext{Args: []string{"dry-run", blocksFile}})
    require.True(t, res.Success)
    require.Contains(t, res.Output, "diff-text")
}

func TestEditCommand_Commit_ValidPrompt_DelegatesCommit(t
    *testing.T) { /* analogous; assert commitFn called */ }
func TestEditCommand_Usage_OnUnknownSub(t *testing.T) {
    c := NewEditCommand(&fakeInspector{})
    res, _ := c.Execute(context.Background(),
        &CommandContext{Args: []string{"explode"}})
    require.False(t, res.Success)
    require.Contains(t, res.Output, "/edit")
}

```

Subject: feat(P1-F17-T07): /edit slash command (status/diff/dry-run/commit) + SmartEditInspector.

---

## Task 8: main.go wiring + registry registration + integration tests

**Files:** modify `helix_code/cmd/cli/main.go`, modify `helix_code/internal/tools/registry.go`, new `helix_code/tests/integration/smartedit_test.go` (`//go:build integration`).

`main.go` additions (alongside the existing `multiedit.NewMultiFileEditor(...)` call):

```
import "dev.helix.code/internal/tools/smartedit"

// after multiedit editor is constructed (already named `mfe` in
    main):
smartTool := smartedit.NewSmartEditTool(mfe, logger)
toolReg.Register(smartTool)
slashRegistry.Register(commands.NewEditCommand(smartTool))
```

`registry.go` — register `smart_edit` in the existing tool init block alongside `multiedit_commit` (no new fields, no new setters).

Integration tests (`-tags=integration`; `ALWAYS-RUN`, no infra dep):

```
//go:build integration
// +build integration

func TestSmartEdit_SingleFile_ContentChanged(t
    *testing.T) { /* sha-before != sha-after */ }
func TestSmartEdit_SearchNotFound_FileUnchanged(t
    *testing.T) { /* sha-before == sha-after */ }
func TestSmartEdit_MultiFile_AllChanged(t
    *testing.T) { /* 3 files, all sha changed
    */ }
func TestSmartEdit_MidCommitFailure_AllFilesUnchanged(t
    *testing.T) {
    if runtime.GOOS == "windows" {

        t.Skip("SKIP-OK: P1-F17 chmod-rollback test requires unix
            permission semantics (Windows)")
    }
    // chmod 0500 on parent dir of file 3; assert all 5 files
        byte-identical to pre-state
}
func TestSmartEdit_DiffMatchesDiffU(t
    *testing.T) { /* compare against `diff
    -u` via os/exec */ }
func TestSmartEdit_Ambiguous_FileUnchanged(t
    *testing.T) { /* 3× SEARCH literal; sha
    unchanged */ }
```

```

func TestSmartEdit_Binary_Refused(t
    *testing.T)                                { /* NUL in first 8 KiB;
    sha unchanged */ }
func TestSmartEdit_DryRun_NeverWrites(t
    *testing.T)                                { /* sha unchanged */ }
func TestSmartEdit_OrderedBlocks_LaterSeesEarlier(t
    *testing.T)                                { /* A→B then B→C; final = C */ }

```

Subject: feat(P1-F17-T08): wire SmartEditTool + /edit slash into main + always-run integration tests.

---

## Task 9: Challenge harness (7-phase, sha-256 positive evidence)

**Files:** new helix\_code/tests/integration/cmd/p1f17\_challenge/main.go, new challenges/p1-f17-smart-file-editing/CHALLENGE.md, new challenges/p1-f17-smart-file-editing/run.sh.

Harness phases (per spec §6.3):

1. **SINGLE-FILE-SUCCESS (always runs)** — tempdir; write sample.go; capture sha-before; run tool with one block; assert Applied=true AND sha-after differs AND result.Files[0].NewContent matches os.ReadFile(sample.go) byte-for-byte AND result.Files[0].Diff contains @@.
2. **SEARCH-NOT-FOUND-REJECTED (always runs)** — tempdir; write unchanged.go; capture sha-before; run tool with deliberately-unfindable SEARCH; assert tool returned ErrSearchNotFound AND sha-after equals sha-before (file untouched).
3. **MULTI-FILE-ATOMIC (always runs)** — 5 files; capture sha-before for each; run tool with 5 blocks; assert Applied=true AND every sha-after differs from sha-before.
4. **PARTIAL-FAILURE-ROLLBACK (always runs; gated on unix permissions)** — 5 files; capture sha-before for each; chmod 0500 on parent dir of file 3 (force write failure); run tool with 5 blocks; assert tool returned commit error AND every sha-after EQUALS sha-before (atomic rollback). Cleanup restores 0700.
5. **DIFF-EXACTNESS (always runs)** — apply a known SEARCH/REPLACE; independently invoke diff -u pre.txt post.txt via exec.CommandContext; assert tool's Diff matches the system output (modulo trailing-newline normalisation).
6. **AMBIGUOUS-REJECTED (always runs)** — file with SEARCH literal repeated 3 times; capture sha-before; run tool; assert ErrSearchAmbiguous AND sha-after equals sha-before.
7. **BINARY-REFUSED (always runs)** — write bin.dat with NUL byte at offset 16; capture sha-before; run tool; assert ErrBinaryFile AND sha-after equals sha-before.

Output skeleton (verbatim per spec §6.3) ends with:

SUMMARY: SINGLE=6/6 PASS; NOT-FOUND=4/4 PASS; MULTI=5/5 PASS; ROLLBACK=6/6

The Challenge MUST exit non-zero on any assertion failure. SHA-256 mismatches (Applied=true but file unchanged, or Applied=false but file changed) are hard failures. Anti-bluff smoke clean check appended to harness output. Verbatim output captured into 06\_phase\_1\_evidence.md. Dual commit (Challenges submodule + meta-repo bump).

Subject: feat(P1-F17-T09): challenge with sha-256 positive evidence per phase (7 always-run phases).

---

## Task 10: Close-out + push

Tick all 10 items in PROGRESS, advance PROGRESS focus to F18 candidate, run final verification:

```
cd HelixCode && make verify-compile
grep -rn "simulated\\|for now\\|TODO implement\\|placeholder" \
    internal/tools/smartedit internal/commands/edit_command.go &&
    echo BLUFF || echo clean
go test -count=1 ./internal/tools/smartedit/... ./internal/
    commands/...
go test -count=1 -tags=integration ./tests/integration/...
go mod tidy && git diff --exit-code go.mod go.sum # MUST be no-
    op (no new deps)
```

Commit chore(P1-F17-T10): close out feature 17 — smart file editing. Push 4 remotes non-force (origin, helixdev, vasic-digital, gitlab per programme conventions). Request explicit user authorization at this step (CONST-043).

---

## Self-review notes

1. **Spec coverage:** every spec section maps to a task — T02 types (§3.3), T03 parser (§4.2), T04 applier + binary detect (§4.3, §5.1), T05 diff (§3.6, §11 #9), T06 SmartEditTool (§4.1, §5.2 criteria 1+2+3+4+8), T07 slash (§3.4, §4.4), T08 main.go wiring + integration tests (§3.6, §6.2), T09 Challenge seven phases (§5.2, §6.3), T10 close-out (§9).
2. **TDD:** every code task starts with a failing test. Parser/applier/diff use pure-function tests against in-memory strings; SmartEditTool tests always go through real tempdirs with `t.TempDir()` + `os.WriteFile/os.ReadFile`; the `multiedit.MultiFileEditor` is the real production type, not a mock. The slash uses a hand-rolled fakeInspector (CONST-035 allows interface fakes for slash tests; the production tool is real).
3. **Type consistency:** `EditBlock`, `EditPlan`, `BlockResult`, `EditResult`, `SmartEditResult`, `SmartEditTool`, `SmartEditInspector`, marker constants, error sentinels, size limits — all match across spec §3.3 and plan T02–T07.
4. **Zero new external deps:** `stdlib` + existing `testify/zap`. `go mod tidy` after T02 produces no new entries in `go.sum`. T10's verification step asserts this loudly.
5. **Anti-bluff (§5.2):** Challenge has SEVEN phases, all always-run. Every phase records sha-256 before and after. The eight real-execution criteria (`Applied=true` ⇒ disk changed; `Applied=false` ⇒ disk unchanged; multi-file atomic; partial-failure rollback; diff exactness; ambiguous rejected; binary refused; no diff text at INFO log level) each have dedicated unit + integration + Challenge assertions. Disk-state mismatch is a hard Challenge failure.
6. **CONST-042:** full diff text is logged at DEBUG only; INFO-level logs carry only diff byte-count. `TestSmartEditTool_DoesNotLogDiffAtInfo` enforces this with `zaptest/observer`. The Challenge's saved evidence file records sha-256 + line counts, never diff text.
7. **CONST-043:** stays on `main`, non-force to all four remotes; explicit user authorization is requested at T10 before pushing.

8. **Lenient conflict detection (Q4=B) — non-obvious call** (recorded in spec §11 #3): re-search the SEARCH literal at apply time; ambiguous (n>1) and not-found (n=0) both fail with clear errors. No mtime/hash strict pre-check. Reason: external tools (gofmt, prettier) reformat files frequently; a strict mtime check would fail every legitimate edit when an unrelated reformat happened in the millisecond between the LLM's plan and the apply.
9. **All-or-nothing across the whole prompt (Q3=B) — non-obvious call**: Phase-1 abort gate runs BEFORE the multiedit transaction opens. If any block on any file fails the in-memory apply, NO files are written. Multi-file atomicity is inherited from multiedit; whole-prompt atomicity is enforced by the smart-edit tool itself.
10. **Diff is string, not \*Diff — non-obvious call** (recorded in spec §11 #1): the `EditResult.Diff` and `SmartEditResult.Diff` fields are plain strings. Reason: lowest-coupling shape for LLM consumers; the structural diff is still available to anyone who calls `multiedit.DiffManager.GeneratedDiff` directly.
11. **No marker-escape mechanism in v1** (recorded in spec §5.3 + §11 #5): a SEARCH or REPLACE that contains the literal marker triplet at column 0 will be parsed as malformed. `TestParser_RejectsCol0MarkerInsideBlock_DocumentingLimitation` pins this so a future v2 can introduce escaping deliberately. The v1 workaround is "indent the marker literal, or use `fs_write`".
12. **No multi-attempt history; only lastResult** (recorded in spec §11 #6): F11 session resume already persists the full agent transcript including tool invocations; smart-edit history is recoverable from the session, not from a parallel in-tool history. v1 keeps the in-tool state minimal.
13. **Re-use multiedit's diff, not a new diff impl** (recorded in spec §11 #9): one LCS-based unified-diff implementation in the codebase, one set of tests, one performance characteristic. The smart-edit `diff.go` is a 5-line wrapper around `multiedit.DiffManager.GeneratedDiff`.
14. **Why a tool + slash but no cobra (Q5=A)**: the LLM's primary surface is the tool; the human's primary surface is the slash. A cobra subcommand `helixcode edit ...` would duplicate the slash without adding value; scripted use is covered by `helixcode --slash-command "/edit dry-run blocks.txt"`.